Hashiko

Distributed File Integrity Monitoring System

Table of Contents

Document Control

Document Information

	Information
Document Owner	Jackie Ma
Document Owner ID	A00889988
Issue Date	2025-09-19
Last Saved Date	2026-03-13
File Name	comp8900_jackie_ma_proposal_final_updated.pdf

Document History

Version	Issue Date	Changes
1.0	2025-09-19	Initiation for Proposal Draft 1
2.0	2025-10-26	Add List of Abbreviations and Scope and Depth, update all sections with provided feedback and add references for Proposal Draft 2
3.0	2025-11-09	Update sections with provided feedback, rewrite backend to Flask and add composite indexes for Milestone 1.
4.0	2025-12-02	Update Methodology and Design, Schedule and Milestones section
5.0	2026-02-08	Update proposal to remove temporal correlation logic.
6.0	2026-02-13	Rename 8800 to 8900.

List of Abbreviations

AIDE - Advanced Intrusion Detection Environment
API - Application Programming Interface
BCIT - British Columbia Institute of Technology
BRIN - Block Range Index (PostgreSQL index type)
CI/CD - Continuous Integration/Continuous Deployment
CRUD - Create, Read, Update, Delete
CSV - Comma-Separated Values
ERD - Entity Relationship Diagram
FIM - File Integrity Monitoring
HIDS - Host-based Intrusion Detection System
HTML - HyperText Markup Language
HTTP - HyperText Transfer Protocol
HTTPS - HyperText Transfer Protocol Secure
ID - Identifier
IDS - Intrusion Detection System
IP - Internet Protocol
IT - Information Technology
JSON - JavaScript Object Notation
JSONB - JSON Binary (PostgreSQL data type)
JWT - JSON Web Token
mTLS - Mutual Transport Layer Security
NATS - Neural Autonomic Transport System (message queue)
NTP - Network Time Protocol
OS - Operating System
OSSEC - Open Source HIDS Security
PDF - Portable Document Format
REST - Representational State Transfer
SHA-256 - Secure Hash Algorithm 256-bit
SIEM - Security Information and Event Management
SQL - Structured Query Language
TLS - Transport Layer Security
UI - User Interface
URL - Uniform Resource Locator
UTC - Coordinated Universal Time

Abstract

Hashiko is a distributed monitoring system that tracks file integrity and user activity across multiple endpoints. Agents collect file hashes, login events and send it to a central server. This agent-server design enables centralized monitoring across distributed systems. The server stores events, validates integrity and generates alerts when suspicious changes are detected. A web interface shows hashes, real-time alerts and event timelines. The system focuses on accuracy, secure transfer and scalability. Testing uses synthetic data, log replays and simulations to validate components. Scope includes agents, server, storage and interface. Out of scope are malware analysis, snapshots and compliance reporting. Hashiko provides centralized visibility into file integrity across distributed systems.

Student Background

I am a fourth-year student in the Bachelor of Applied Computer Science program at BCIT, specializing in Network Security Applications Development. My academic background includes a Computer Systems Technology Diploma from BCIT, specializing in Web and Mobile development. My technical foundation includes proficiency in multiple programming languages: Python, Java, JavaScript, C#, SQL, Swift and Kotlin. I also work with web technologies including HTML and CSS. Frameworks and platform experiences include .NET, Node.js, Express.js, Flask and Laravel. Database experience spans PostgreSQL, MySQL, MariaDB, SQLite and Firebase.

Through previous coursework and personal projects, I have developed several full-stack, backend and database-driven applications. For example, PromptVisioQuiz fetches data from external APIs (CBC RSS feed) and processes it through a multi-service architecture with Node.js/Express and Python/Flask microservices, using JWT authentication and PostgreSQL for data persistence. MealPlanIQ demonstrates my ability to build complex data-driven applications with TypeScript/Angular frontend and Python/Flask backend, managing relational data with PostgreSQL and deploying to cloud infrastructure (GoogleCloud Platform). These projects gave me practical experience in distributed system architecture, RESTful API design, secure authentication, database management andcross-service communication—skills directly applicable to Hashiko's agent-server architecture, event processing pipeline and temporal data storage requirements.

To prepare for this project, I researched file integrity monitoring techniques and distributed security architectures. I read Kaczmarek and Wrobel's analysis of Modern Approaches to File System Integrity Checking approaches to understand current detection methods and their limitations. Their review of user-space and kernel-space integrity checkers helped me understand the trade-offs between periodic checking and real-time monitoring approaches. I also read Abdullah et al.'s work on File Integrity Monitor Scheduling Based on File Security Level Classification scheduling. Their research on combining off-line and on-line monitoring informed my approach to resource prioritization. Beyond these foundational papers, I have explored existing distributed monitoring tools such as OSSEC and Wazuh to understand

agent-server communication patterns. I am currently investigating time-series databases for efficient event storage and message queue systems for reliable data transmission between agents and the central server.

Before studying computer science, I studied architecture and briefly worked in the industry. This background has strengthened my analytical and problem-solving abilities. Examples of my work, such as PromptVisioQuiz and MeanPlanIQ, can be found at www.jackiema.ca. After graduation, I plan to work in software development or IT.

Project Summary

This project is motivated by the growing need for centralized visibility and consistent monitoring in modern distributed systems. As organizations deploy security tools across multiple endpoints, maintaining oversight becomes increasingly difficult. Current security monitoring tools create dangerous blind spots. Traditional file integrity monitors operate independently on each system. They cannot provide unified visibility across the infrastructure. A file modification on one server and a configuration change on another might each seem normal alone. Without centralized monitoring, threats slip through undetected. Organizations need a way to see activity patterns across all systems from a single interface.

Hashiko is a distributed monitoring system that tracks both file integrity and user activity. The project implements a scalable monitoring framework that aggregates events from multiple endpoints into a central view. It goes beyond standalone monitoring tools by providing unified visibility across distributed infrastructure. The focus is on centralized collection and reporting. For example, file changes and login activity are collected from all monitored systems and stored in one location.

Endpoint agents collect file hashes, login sessions and process activity. All data is sent with timestamps to a central server. The server stores events and generates alerts when integrity violations are detected. Alerts trigger when file hashes change or suspicious activity matches predefined patterns. A web interface provides clear visibility, easy alert tracking and quick access to detailed event data. The administrator has a centralized and readable view of ongoing activity.

The goal is to enable centralized monitoring that standalone tools cannot provide. By collecting events from all endpoints into a unified system, administrators gain complete infrastructure visibility. Expected outcomes include faster breach detection as changes are immediately visible in the central dashboard. False positives will decrease because administrators can correlate activity across systems manually. Security teams will investigate faster using timeline views that show all endpoint activity. The system transforms isolated monitoring into unified visibility. This provides organizations with actionable insights and significantly improved protection through centralized oversight.

Problem Statement and Background

Organizations face increasing risks from both external attackers and insider threats. Traditional tools such as intrusion detection systems or file integrity monitors can detect single events. Tools like Tripwire and OSSEC use hash comparison to detect file changes [1,2]. AIDE, Samhain and Wazuh also perform file integrity monitoring. AIDE and Samhain use scheduled hash checks. Wazuh extends OSSEC with central logging and rule-based alerts. These tools detect isolated events on individual systems. They do not provide unified visibility across distributed infrastructure. Studies by Abdullah et al. and Kaczmarek and Wrobel confirm that periodic and user-space methods delay detection [1,2]. Their periodic checking creates detection delays and higher inspection frequency improves detection but degrades performance. User-space integrity checkers can only verify if files were modified between checking operations [1]. During this period, attackers can replace system files with backdoors or trojans that may be executed by authorized users [1]. However, they rarely provide centralized management across multiple endpoints. This leaves blind spots where administrators lack visibility into distributed system activity. For example, file changes occurring across multiple servers may seem harmless when viewed in isolation but require centralized oversight for effective monitoring.

The core weakness is the lack of centralized visibility in existing monitoring systems. Unified oversight across endpoints is as important as detecting changes themselves. Traditional FIM tools face a fundamental trade-off. High-frequency inspection maintains effectiveness but hurts performance. Low-frequency inspection reduces overhead but allows threats to persist undetected. Manual policy configuration becomes impractical in distributed environments [2]. Without centralized management, significant risks bypass defences. SIEM tools like Splunk and Elastic Security try to improve visibility. They collect and aggregate logs from multiple sources. However, they focus on log analysis rather than real-time file integrity monitoring. Centralized file integrity monitoring is still missing. Visibility is further reduced by scattered monitoring tools and fragmented dashboards that slow incident response. A centralized system that aggregates events from multiple endpoints and stores them in one location can close this visibility gap. Centralized monitoring is essential for security. Attackers with administrator privileges can compromise local security tools. Remote monitoring from a privileged server provides protection even when the monitored system is compromised [2]. A web interface adds readability by simplifying alert tracking and management.

Hashiko closes this gap. It uses agents that collect events and send them to a central server. The server stores all activity in one location for unified visibility. This method combines the reach of SIEM tools with the detail of host monitors. It delivers centralized monitoring instead of isolated standalone tools.

Inventiveness

What sets Hashiko apart is its focus on centralized multi-endpoint monitoring. Most security tools monitor single systems in isolation, such as one server's file changes or one host's login activity. The project instead collects data from multiple endpoints into a unified system. It provides visibility that becomes useful only when activity from different systems can be viewed together in one interface. Existing studies focus on improving detection accuracy through frequency tuning and signature updates [1,2]. Few examine centralized collection architectures that combine file integrity with authentication and process monitoring. This unified monitoring approach remains underexplored in host-based intrusion detection research.

Another inventive aspect is the centralized distributed design. Endpoint agents collect activity data such as file hashes, logins and processes. This follows established HIDS deployment patterns. However, it extends beyond traditional file checking. It incorporates authentication and privilege events into the same data stream. They then attach precise timestamps before sending it to the central server. Instead of producing isolated alerts on each endpoint, the system aggregates all events centrally. A web interface presents this data in a clear way, giving teams both unified oversight and visibility. This blend of distributed data gathering with centralized aggregation is rarely seen in current monitoring solutions. While some commercial SIEM systems such as Splunk Enterprise Security and IBM QRadar provide event aggregation, they focus on log collection rather than real-time file integrity monitoring. Academic systems like I3FS and SOFFIC achieve real-time monitoring but require kernel-level modifications and limited portability. Hashiko's user-space approach provides cross-platform deployment without kernel dependencies, combining practicality with comprehensive event collection.

Traditional FIM solutions require manual policy configuration. Administrators must specify which files to monitor and how often. This becomes impractical in large-scale environments. Hashiko simplifies management through centralized configuration and alerting. Unlike systems that monitor only files or logs, Hashiko collects multiple event types in one system. Traditional file integrity checkers like Tripwire, AIDE and AFICK work similarly—they all perform periodic hash comparisons on individual systems. Hashiko differs by collecting file modifications alongside user activity data rather than treating each system as an isolated monitoring target. This provides unified visibility that individual tools would miss.

Complexity

The project is complex as the system must collect data from many endpoints and aggregate events in real time with precise timestamps. This makes it significantly more advanced than simple standalone monitoring. The system integrates distributed agents, secure real-time communication, centralized event storage and a unified web interface. The most difficult aspect is managing secure data collection from multiple distributed endpoints, which is far more challenging than monitoring a single system. Success requires skills in distributed systems, secure communication protocols and scalable database design. This project requires

independent design decisions that go beyond coursework. Trade-offs between monitoring frequency and system performance must be experimentally evaluated.

The system requires designing efficient data ingestion pipelines that handle concurrent event streams from multiple agents. This exceeds traditional FIM approaches. Tools like Tripwire perform simple hash comparisons on individual systems, while centralized monitoring requires handling concurrent data streams from distributed sources. Traditional integrity checkers simply compare current hash values with baseline patterns stored in a secure database. Hashiko requires managing agent authentication, handling network failures gracefully and efficient querying across large event datasets. Conventional integrity monitoring tools do not address these distributed system challenges. Implementing secure agent-server communication requires understanding encryption, certificate management and retry logic. No off-the-shelf solution exists for distributed file integrity monitoring with this architecture. The system integrates distributed architecture, real-time event processing and security monitoring into a cohesive platform. This combination of distributed data collection, secure communication and centralized storage represents Bachelor-level integration. The project demands trade-off analysis between monitoring frequency and network overhead.

Technical Challenges

Distributed agents must be designed to securely collect and transmit data. Authentication mechanisms are needed to prevent agent spoofing. File integrity databases must be secured because if an attacker modifies the database, the entire security system becomes unreliable. Most integrity checkers store databases on read-only devices or use cryptographic functions to protect them. Secure agent-server communication is critical. This is especially true when agents operate on potentially compromised systems. Event data must use encrypted channels to prevent interception. Network interruptions must be handled without losing events. Ensuring exactly-once delivery requires handling network failures and retries without creating duplicates. Message acknowledgments must be tracked with unique IDs. Failed transmissions will require retry logic with exponential backoff to prevent network congestion. Learning message queue systems like NATS is necessary to implement these delivery guarantees. Clock synchronization across distributed endpoints is challenging due to network latency and drift. NTP synchronization will be used to ensure consistent timestamps across all agents. This timing challenge is more complex than traditional FIM. Off-line tools like AIDE schedule periodic checks at fixed intervals. Real-time collection requires precise timestamp synchronization. This ensures accurate event logging from distributed systems. Learning NTP client libraries and designing clock drift detection mechanisms is necessary.

A central server will be required to store and process large event streams in real time. It must handle concurrent data ingestion and scaling across multiple hosts. The storage system must manage high-velocity writes from distributed agents. For example, receiving file integrity events, login records and process data from dozens of endpoints simultaneously without overwhelming the server. Storage must support fast queries across multiple dimensions. Traditional FIM tools store baseline databases that are updated periodically, requiring significant maintenance

overhead. Hashiko's model differs fundamentally by supporting high-velocity event ingestion and enabling complex queries across multiple dimensions. These include time range, endpoint, user and file path while ingesting thousands of events per second. This requires composite indexes and table partitioning. Learning PostgreSQL's declarative partitioning and indexing strategies is required for query optimization.

The web interface presents challenges in real-time state management and alert visualization. Learning React with WebSocket integration is necessary for live alert updates. The dashboard must display events from multiple agents in a unified view, requiring understanding of state management and efficient rendering. Centralized monitoring techniques from industry research must be studied and adapted to the FIM context. Multiple storage and indexing approaches will be prototyped and benchmarked to determine optimal performance trade-offs between write throughput and query speed.

Methodology and Design

The project follows an agent-collector-storage design with a central server and web interface. This architecture separates data collection from storage and presentation. This pattern appears in multi-platform HIDS deployments. Centralized management enables efficient monitoring of multiple systems. It addresses scalability limitations of standalone FIM tools. Standalone file integrity checkers like Tripwire were originally designed to work independently before becoming integrated into comprehensive intrusion detection systems. Hashiko's architecture follows this evolution by integrating file integrity monitoring with authentication and privilege monitoring from the start. Agents compute file hashes, monitor file paths, record logins and send timestamped events. The server ingests, validates and stores events. The server aggregates all activity into a centralized database for unified visibility. The web interface delivers real-time alerts, event timelines and configuration management for rapid triage and investigation.

System Architecture

Figure 1 illustrates Hashiko's architecture. Endpoints run Linux agents that monitor file changes, logins and processes. Agents synchronize time via NTP and transmit timestamped events with hashes to the Ingest API via HTTP POST. The Normalizer validates data and applies UTC conversion to ensure consistent timestamps across distributed endpoints. Normalized events are stored in the central database, which maintains file integrity baselines and event history. When file hashes change or suspicious activity is detected, the Alert Service generates alerts with severity and metadata. The web interface provides real-time dashboards via WebSocket, a REST API for programmatic access and a Configuration Manager for system settings. Storage uses an append-only Event Store for immutability, versioned configuration files for change tracking and composite indexes on time, endpoint and path for efficient queries. External integrations enable alert forwarding to ticketing systems and annotations for incident tracking.

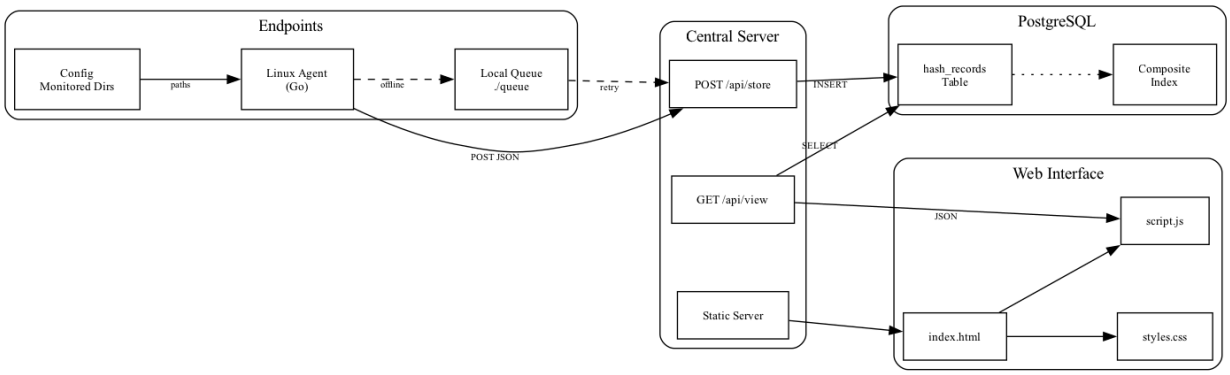

Figure 1 - System Diagram

Agent Architecture

The agent uses SHA-256 hashing via crypto/sha256. SHA-256 is a cryptographic hash function that produces fixed-size output regardless of input length. This ensures that even a single bit change in a file generates a different hash value. The hashDirectory function walks directories with filepath.Walk and hashes file paths and contents. Five directories are monitored: /boot, /usr/bin, /usr/sbin, /etc and /root. These directories contain common attack targets [2]. System files control core operating system functions. Malicious modifications can disrupt services. They can enable attacks on other systems [2]. Excluding volatile paths like cache reduces false positives. It also reduces performance overhead. Volatile paths like .cache, .mozilla, tmp and session files are filtered. JSON output includes timestamp and directory hashes. Transmission via HTTP POST to /api/store.

Server Architecture

Built with Flask and PostgreSQL. Loads environment variables and creates a hash_records table with id, timestamp and five hash columns indexed by id descending. The handleStore endpoint decodes JSON, normalizes client IP and inserts records with parameterized queries. The handleView endpoint returns all records as JSON ordered newest first. Serves static files on port 8800.

Development

Development will follow an incremental approach. Milestone 1 implements agent data collection, basic server storage, SHA-256 hashing, HTTPS and composite indexes. Milestone 2 implements message queuing and retry logic. Milestone 3 implements admin login, hash comparison and alert system. Milestone 4 implements individual file hashes, update agent_id and deployment. Milestone 5 includes tests and documentation. Each milestone includes integration and system testing before proceeding. Version control with Git enables rollback if issues arise.

Scalability

This modular design enables horizontal scaling. Multiple server instances can process events concurrently using a shared PostgreSQL database. Weekly partitioning allows old partitions to

be archived without impacting active queries. Composite indexes on (timestamp, agent_id, event_type) enable query parallelization across partitions.

Interface
The HTML landing page shows an admin login page. Once logged in, the table shows agent ID, timestamp and hash columns. JavaScript fetches from the backend and populates table rows. Alert notifications and logout button is located in the header.

Database Schema
The hash_records table has auto-incrementing id, text timestamp and hash columns for SHA-256 strings. Index on id descending optimizes queries. Limitations include no agent tracking, no event types and text timestamps that prevent range queries. The planned migration addresses these shortcomings.

Planned Database Migration
Three tables will replace hash_records. Events table stores event_id, agent_id, timestamp (proper timestamp type), event_type (login, file_modification, process_execution), payload (JSONB with event details) and hash_value. Alerts table contains alert_id, timestamp and event_id. Composite indexes on timestamp, agent_id and event_type enable fast queries. Weekly partitioning manages growth.

This three-table design separates concerns effectively. It improves upon single-table FIM implementations. The separation enables centralized agent management and efficient event storage. Traditional FIM tools store all data in monolithic structures. The normalized schema enables better query performance and scalability. Existing tools lack this architectural separation.

Planned Enhancements
Activity monitoring for logins (/var/log/auth.log) and process execution. Extended JSON schema with event_type, agent_id and ntp_offset_ms. NTP syncs every 300 seconds with drift flagging over 1000ms. Alert generation based on file integrity violations and configurable thresholds. Three-table schema: events, agents and alerts. Composite indexes on (timestamp, agent_id, event_type) with weekly partitioning and 90-day retention.

Maintainability
The four-table schema improves maintainability over single-table FIM. Rules can be modified without schema changes. New event types require only payload JSONB updates. Agent registration is independent of event storage. This separation enables independent component updates and testing.

Test Plan

The project will be tested in a staged lab. Synthetic generators, safe log replays and controlled adversarial simulations will be used. This addresses a key limitation in FIM research. Evaluating

detection effectiveness is difficult without compromising production systems. File integrity checkers are typically tested either through periodic manual execution by administrators or through scheduled runs using services like Cron. Hashiko's testing approach simulates attack scenarios rather than waiting for periodic checks. Controlled simulations enable safe validation. These tests will validate agents, transfer, normalization, correlation, storage, APIs and the web interface. Verification uses clear checkpoints for accuracy, throughput, detection latency, durability and UI responsiveness. A release moves forward only when all checkpoints are passed.

Unit Testing
Go testing package with testify verifies agent hash computation accuracy, hash stability across runs, change detection, skip path filtering and transmission success. Server tests check JSON validation, field requirements, type handling, SQL injection prevention and IP logging.

Integration Testing
End-to-end test runs server and agent in separate processes. Agent transmits data. Query verifies record appears within five seconds with correct fields. Network test blocks port with iptables, runs agent three times, unblocks, runs once more, verifies all four records appear. Future correlation test injects timed events and verifies alert generation.

Performance Testing
The load test runs 10 agents every 30 seconds for 10 minutes. Measures response time (target p95 under 200ms), throughput (over 20/second) and memory (under 100MB). Stress test scales to 100 agents finding breaking point. Query test measures response time with 1000 to 100,000 records (target under 500ms).

System Testing
File integrity validation includes monitoring /etc/passwd for unauthorized modifications. The /etc/passwd file controls user authentication. Modification by any process should trigger an alert. This test validates Hashiko's file integrity monitoring capabilities. Unauthorized access simulation includes detecting changes to system binaries in /usr/bin and /usr/sbin. Mass modification simulation includes 20 rapid file changes across monitored directories. Current implementation detects hash changes. Future versions will generate severity-based alerts with detailed event logs.

These scenarios simulate real-world attack patterns. Adversaries modify system files to establish persistence or escalate privileges. Traditional FIM would detect the file modification after the next scheduled check. User-space integrity checkers can only detect that a file changed, not prevent the execution of compromised files [1]. They do not protect the file system in real-time [1]. Traditional FIM running on individual systems lacks centralized visibility across multiple endpoints. Hashiko's centralized architecture provides unified monitoring that helps administrators track file integrity across distributed infrastructure and should detect modifications on any monitored endpoint.

Test Environment
Main testing on desktop with Fedora 42, Python 3.14 and Go 1.24.8. Secondary testing on MacBook M1 Air for cross-platform validation. Future testing on school lab machines for multi-endpoint testing with Linux and Windows.

Test Data Management
Test data includes synthetic event logs with known attack patterns, baseline hash databases for integrity checking and timed event sequences for correlation validation. Controlled datasets enable reproducible testing and regression detection. Test data versioning ensures consistent results across test runs.

Success Metrics
Over 80% code coverage (current 0%), under 1% integration errors, zero false negatives on file integrity (current: /etc, /boot, /usr/bin, /usr/sbin, /root; future: temporal patterns), under 5% false positives, performance targets met, no critical/high bugs. Testing in two-week sprints with regression. Unit tests run first (fail fast), followed by integration tests if units pass. Failed tests block merges, preventing broken code from entering the main branch. Weekly system and performance tests. Final 48-hour soak test with 100 agents at 60-second intervals validates stability and resource usage. When tests fail, root cause analysis identifies whether issues stem from code defects, test design flaws or environmental factors. Critical failures halt development until resolved. Non-critical issues are tracked as technical debt and addressed in future sprints.

Expected Outcomes
- The system detects unauthorized file modifications.
- Alerts are generated when rule conditions are matched.
- No events are lost during network interruptions.
- All significant test cases result in measurable logs and clear status (pass/fail).
- False positives are minimized during normal (benign) usage.

Sample Test Cases
- Detect file modification:
    - Modify a file in /usr/bin on a monitored agent.
    - Expect the system to detect the hash change and generate an alert.
- Replay normal activity log:
    - Feed benign file writes to monitored directories.
    - Expect zero alerts; confirms low false positive rate.
- Network failure simulation:
    - Disconnect agent network during events, then reconnect.
    - Expect events are queued, delivered after reconnect and the system state matches the test input (no lost/duplicate events).
- Multi-agent monitoring:

- Generate file modification events from three separate agents simultaneously.
- Expect all events to be received, stored and displayed correctly in the web interface.

Scope and Depth

Scope includes agents, a central server, storage and a web interface. Features cover file integrity with baselines and hashes, login and process monitoring, normalized timestamped events, secure transfer, alerts, search and configuration management. Hashiko employs hash-based integrity checking similar to Tripwire and OSSEC. However, it extends beyond standalone monitoring by incorporating centralized collection from multiple distributed endpoints. A file modification is recorded with full context including agent identity, timestamp and event metadata. It is not just an isolated hash comparison. Deliverables are another OS agent, a central pipeline, indexed storage, web interface, documented APIs, baseline configuration and a test plan.

Out of scope are memory scanning, kernel exploit detection, malware analysis and compliance reports. These exclusions distinguish Hashiko from comprehensive HIDS platforms like OSSEC. OSSEC includes rootkit detection and compliance reporting. Hashiko's focused scope enables deeper implementation of distributed monitoring architecture. It does not attempt to replicate full HIDS functionality.

Depth comes from accurate time sync, centralized aggregation, scalable ingest, durable storage with replay capability, configurable alerts, unified dashboards, baseline management and full security for identity, encryption and audit. This depth differentiates Hashiko from traditional FIM. Tools like Samhain and AIDE provide centralized management and periodic checking but lack real-time collection and unified web interfaces. Some advanced systems like I3FS and SOFFIC operate at the kernel level to provide real-time file checking. However, these require kernel modifications and are platform-specific. Hashiko takes a different approach—using user-space agents with centralized storage rather than kernel integration. Hashiko addresses the performance-versus-effectiveness trade-off. It focuses resources on secure data collection rather than continuous file hashing.

Schedule and Milestones

This project spans two academic terms. It covers both COMP 8800 and COMP 8900. The following tables show a detailed week-by-week schedule. The plan outlines all key project milestones. This includes research, implementation and testing.

COMP 8800 focuses on core features. The goal is a functional prototype. It also includes the finalized project proposal. COMP 8900 builds advanced features. This term focuses on hardening the system. It concludes with the final project submission and presentation. All deliverables align with course milestones.

COMP 8800		
Week	**Key Activities & Focus**	**Due Date / Milestone**
1	Formulate project idea. Draft sections: Student Background, Project Summary.	
2	Draft sections: Inventiveness, Complexity, Technical Challenges.	
3	Draft Problem Statement. Revise Project Summary. Compile sections 1-6.	Proposal Draft 1 Due
4	Build initial prototype: agent hashing, basic server endpoint for data ingest.	
5	Refine prototype. Draft Methodology and Test Plan sections based on findings.	
6	Complete and document prototype for submission. Draft Scope, Schedule and References.	Prototype Due
7	Revise all proposal sections, incorporating insights from the prototype.	Proposal Draft 2 Due
8	Address initial feedback on Draft 2. Begin Milestone 1: Implement composite index and rewrite backend to Flask.	
9	Continue development on Milestone 1 deliverables.	
10	Complete all coding and documentation for Milestone 1.	Milestone 1 Due
11	Review feedback from instructor and second reader. Plan proposal revisions.	
12	Revise proposal based on all feedback. Begin Milestone 2: Implement message queue and retry logic.	
13	Continue development on Milestone 2 deliverables.	
14	Continue Milestone 2. Address proposal feedback.	
15	Integrate all Milestone 2 components. Complete Final Proposal, demo and give a presentation.	Final Proposal & Milestone 2 Due

COMP 8900		
Week	**Key Activities & Focus**	**Due Date / Milestone**
1 - 5	Begin Milestone 3: Implement hash comparison, login and alert system.	Milestone 3 Due
6 - 10	Begin Milestone 4: Implement individual hash files, implement custom watch path, replace agent_id with intuitive name, deploy backend and frontend with TLS.	Milestone 4 Due
11 - 14	Begin Milestone 5: Implement tests and documentation.	Milestone 5 Due
15	Deliver final presentation. Submit all project files.	Final Project Submission

References

[1] J. Kaczmarek and M. Wrobel, “Modern Approaches to File System Integrity Checking,” *2008 1st International Conference on Information Technology*, pp. 1–4, May 2008. doi:10.1109/inftech.2008.4621669

[2] Z. H. Abdullah, N. I. Udzir, R. Mahmod and K. Samsudin, “File Integrity Monitor Scheduling Based on File Security Level Classification,” *Communications in Computer and Information Science*, pp. 177–189, 2011. doi:10.1007/978-3-642-22191-0_16